LuckyDraw Pro

Technical Specification Document

Version 1.0 - MVP Release

Prepared For:

Executive Committee

Prepared By:

System Architecture Team

February 2026

Confidential Document - Internal Use Only

Contents

Executive Summary **3**

1 System Architecture Overview **4**
1.1 Technology Stack 4
1.2 Database Schema Architecture 5
1.2.1 Core Tables Structure 5
1.3 System Data Flow Diagram 7

2 Core Feature Implementation **8**
2.1 Authentication & User Management 8
2.1.1 Feature Matrix 8
2.2 Wallet System Architecture 8
2.2.1 Transaction State Machine 8
2.2.2 Transaction Processing Rules 8
2.3 Draw Engine Implementation 9
2.3.1 Draw State Management 9
2.3.2 Verifiable Winner Selection Algorithm 9

3 Role-Based Access Control **12**
3.1 Permission Matrix 12
3.2 Agent Operations Framework 12
3.2.1 Agent Capabilities 12
3.2.2 Anti-Fraud Safeguards 12

4 Transparency & Fair Play Mechanisms **13**
4.1 Verifiable Draw Protocol 13
4.2 Transparency Features 13

5 Development & Deployment Roadmap **14**
5.1 Phase 1 - MVP Delivery (15-20 Days) 14
5.2 Phase 2 - Enhanced Features (Post-MVP) 14

6 Quality Assurance & Security Framework **15**
6.1 Security Hardening Checklist 15
6.2 Performance Targets 15

7 Compliance & Legal Framework **16**
7.1 Required Documentation 16
7.2 Legal Terminology Guidelines 16

8 Risk Assessment & Mitigation **17**

9 Success Metrics & Key Performance Indicators **18**

10 Deployment & Environment Configuration **19**
10.1 Production Environment Specifications 19
10.2 CI/CD Pipeline 19

11 Conclusion & Next Steps **20**
11.1 Immediate Actions Required 20
11.2 Deliverable Acceptance Criteria 20

Executive Summary

LuckyDraw Pro is a complete raffle management system designed with enterprise-grade architecture and security patterns. Built on Laravel 11, this system implements multi-tier user roles, automated winner selection, and transparent prize distribution mechanisms. The Minimum Viable Product (MVP) focuses on core functionality while maintaining scalability for future enhancements.

Key Deliverables:

- Complete authentication and role-based access control system
- Digital wallet with transaction processing capabilities
- Draw management engine with verifiable winner selection
- Agent operations interface for customer management
- Administrative dashboard for system oversight

Development Timeline: **15-20 days** for MVP delivery

1 System Architecture Overview

1.1 Technology Stack

Component	Technology	Justification
Framework	Laravel 11	Latest LTS, Active security patches
Authentication	Laravel Breeze	Lightweight, Highly customizable
Roles & Permissions	Spatie Laravel-Permission	Industry standard ACL
Database	MySQL 8.0	ACID compliance, Referential integrity
Queue Driver	Redis / Database	Async notification handling
Frontend	Blade + TailwindCSS	Rapid development, Responsive design
API (Future)	Laravel Sanctum	Scalable token management

1.2 Database Schema Architecture

1.2.1 Core Tables Structure

```sql
-- Users & Authentication
CREATE TABLE users (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NULL,
    date_of_birth DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    role_id BIGINT,
    last_login_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_email (email),
    INDEX idx_role (role_id),
    CONSTRAINT fk_user_role FOREIGN KEY (role_id) REFERENCES roles (id)
);
```

Listing 1: Users & Authentication Table

```sql
-- Wallet Management
CREATE TABLE wallets (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT UNIQUE NOT NULL,
    balance DECIMAL(10,2) DEFAULT 0.00,
    lifetime_deposit DECIMAL(10,2) DEFAULT 0.00,
    lifetime_withdrawal DECIMAL(10,2) DEFAULT 0.00,
    last_transaction_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_user_id (user_id),
    CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE transactions (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT NOT NULL,
    wallet_id BIGINT NOT NULL,
    type ENUM('deposit', 'withdrawal', 'ticket_purchase', 'prize_credit') NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    reference_id VARCHAR(36) UNIQUE NOT NULL,
```

```sql
    description VARCHAR(255),
    status ENUM('pending', 'approved', 'rejected', 'completed') DEFAULT 'pending',
    metadata JSON,
    approved_by BIGINT NULL,
    approved_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_user_id (user_id),
    INDEX idx_status (status),
    INDEX idx_reference (reference_id),
    CONSTRAINT fk_transaction_user FOREIGN KEY (user_id) REFERENCES users(id),
    CONSTRAINT fk_transaction_wallet FOREIGN KEY (wallet_id) REFERENCES wallets(id)
);
```

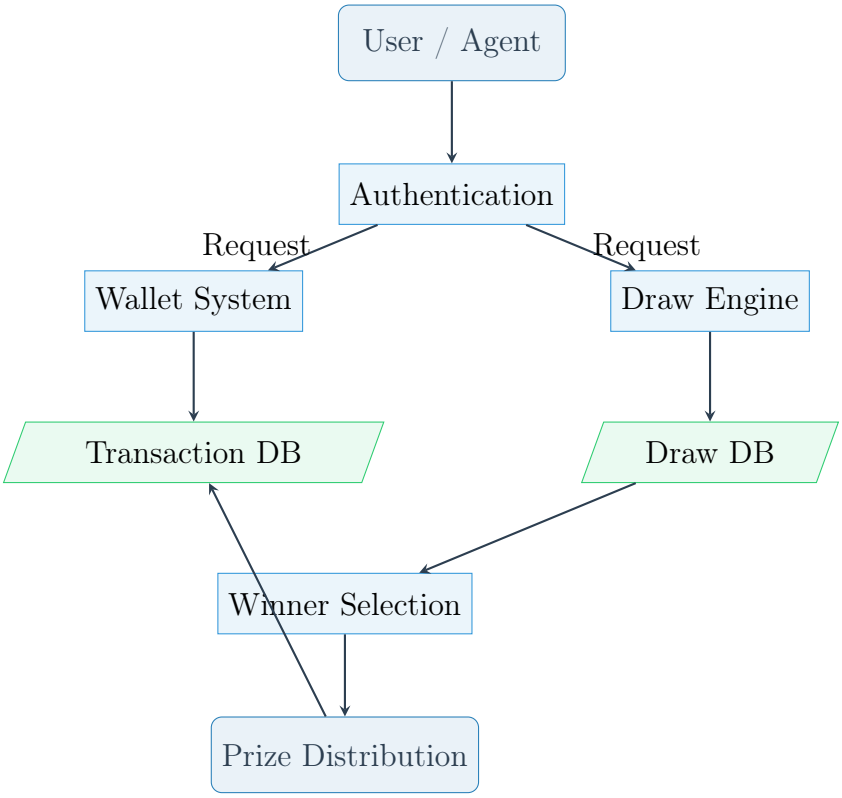
Listing 2: Wallet Management Tables

```sql
-- Draw Management
CREATE TABLE draws (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    description TEXT,
    ticket_price DECIMAL(8,2) NOT NULL,
    total_pool DECIMAL(12,2) DEFAULT 0.00,
    max_tickets INTEGER NULL,
    sold_tickets INTEGER DEFAULT 0,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    draw_time DATETIME NOT NULL,
    status ENUM('draft', 'scheduled', 'live', 'closed', 'drawing', 'completed', 'cancelled')
          DEFAULT 'draft',
    winner_selected_at TIMESTAMP NULL,
    winner_selection_method ENUM('system', 'manual') NULL,
    seed_hash VARCHAR(64) NULL,
    created_by BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_status (status),
    INDEX idx_draw_time (draw_time),
    INDEX idx_created_by (created_by),
    CONSTRAINT fk_draw_creator FOREIGN KEY (created_by) REFERENCES users(id)
);

CREATE TABLE tickets (
```

```sql
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    ticket_number VARCHAR(50) UNIQUE NOT NULL,
    user_id BIGINT NOT NULL,
    draw_id BIGINT NOT NULL,
    transaction_id BIGINT NOT NULL,
    is_winner BOOLEAN DEFAULT FALSE,
    prize_tier_id BIGINT NULL,
    prize_amount DECIMAL(10,2) NULL,
    metadata JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_draw_id (draw_id),
    INDEX idx_user_id (user_id),
    INDEX idx_winner (is_winner),
    INDEX idx_ticket_number (ticket_number),
    UNIQUE KEY unique_draw_ticket (draw_id, ticket_number),
    CONSTRAINT fk_ticket_draw FOREIGN KEY (draw_id) REFERENCES draws(id),
    CONSTRAINT fk_ticket_user FOREIGN KEY (user_id) REFERENCES users(id),
    CONSTRAINT fk_ticket_transaction FOREIGN KEY (transaction_id) REFERENCES transactions(id)
);
```

Listing 3: Draw Management Tables

1.3 System Data Flow Diagram



2 Core Feature Implementation

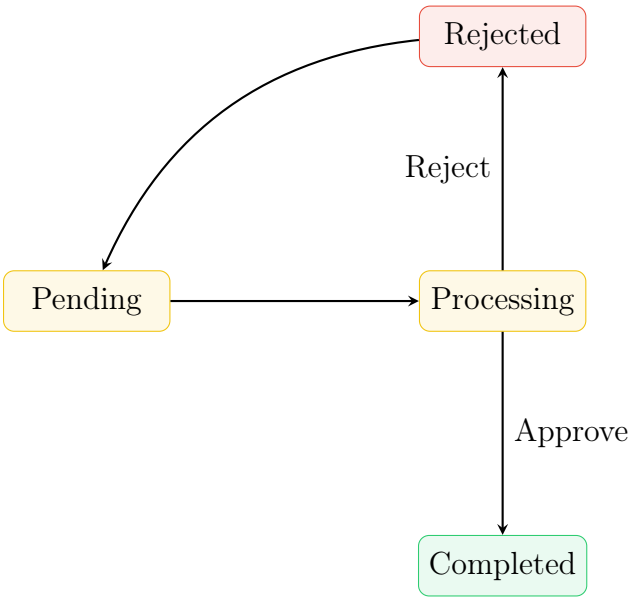
2.1 Authentication & User Management

2.1.1 Feature Matrix

Feature	Implementation Details
Registration	Email/Phone registration with DOB validation (18+)
Login	Multi-factor ready, rate limited (5 attempts/10 minutes)
Password Reset	Email OTP with secure token generation
Account Status	Active/Suspended with reason tracking
Session Management	Device tracking with concurrent session control
Security	Bcrypt hashing, CSRF protection, XSS filtering

2.2 Wallet System Architecture

2.2.1 Transaction State Machine



2.2.2 Transaction Processing Rules

- **Minimum Withdrawal:** BDT 500 (configurable via admin panel)
- **Processing Fee:** 0% (MVP phase)
- **Processing Window:** 24-48 hours
- **Reference Format:** UUID v4 for global uniqueness
- **Audit Requirement:** All status changes logged with timestamp and actor ID

2.3 Draw Engine Implementation

2.3.1 Draw State Management

```php
class DrawStateManager
{
    protected $allowedTransitions = [
        'draft' => ['scheduled', 'cancelled'],
        'scheduled' => ['live', 'cancelled'],
        'live' => ['closed', 'cancelled'],
        'closed' => ['drawing', 'cancelled'],
        'drawing' => ['completed', 'cancelled'],
        'completed' => ['archived'],
        'cancelled' => ['archived']
    ];

    public function transition(Draw $draw, string $newState): bool
    {
        if (!in_array($newState, $this->allowedTransitions[$draw->status])) {
            throw new InvalidStateTransitionException(
                "Cannot transition from {$draw->status} to {$newState}"
            );
        }

        return $draw->update(['status' => $newState]);
    }
}
```

Listing 4: Draw State Transitions

2.3.2 Verifiable Winner Selection Algorithm

```php
<?php

namespace App\Services\Draw;

use App\Models\Draw;
use App\Models\Ticket;
use App\Models\AuditLog;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;

class WinnerSelector
{
    /**
     * Select winner using verifiable random algorithm
     *
     * @param Draw $draw
     * @return Ticket
```

```
     * @throws WinnerSelectionException
     */
    public function selectWinner(Draw $draw): Ticket
    {
        return DB::transaction(function () use ($draw) {
            // 1. Validate draw state
            if ($draw->status !== 'drawing') {
                throw new WinnerSelectionException(
                    "Draw must be in 'drawing' state. Current: {$draw->status}"
                );
            }

            // 2. Generate verifiable seed
            $seed = hash('sha256',
                $draw->id .
                now()->timestamp .
                $draw->sold_tickets .
                config('app.key')
            );

            // 3. Store seed for future verification
            $draw->update(['seed_hash' => $seed]);

            // 4. Seeded random selection
            $ticket = Ticket::where('draw_id', $draw->id)
                ->inRandomOrder($seed)
                ->first();

            if (!$ticket) {
                throw new WinnerSelectionException('No tickets found for this draw');
            }

            // 5. Mark as winner with metadata
            $ticket->update([
                'is_winner' => true,
                'prize_amount' => $draw->total_pool
            ]);

            // 6. Audit trail
            AuditLog::create([
                'user_id' => auth()->id(),
                'action' => 'winner_selected',
                'model_type' => Draw::class,
                'model_id' => $draw->id,
                'metadata' => [
                    'ticket_id' => $ticket->id,
                    'seed_hash' => $seed,
                    'selection_time' => now()->toIso8601String()
                ]
```

```
            ]);

            // 7. Update draw status
            $draw->update([
                'status' => 'completed',
                'winner_selected_at' => now(),
                'winner_selection_method' => 'system'
            ]);

            return $ticket;
        });
    }
}
```

Listing 5: Transparent Winner Selection

3 Role-Based Access Control

3.1 Permission Matrix

Permission	Super Admin	Admin	Agent	User
Create Draw	✔	✔	✖	✖
Select Winner	✔	✔	✖	✖
Approve Withdrawal	✔	✔	✖	✖
Deposit to User	✔	✔	✔	✖
View Reports	✔	✔	✖	✖
Manage Users	✔	✔	✖	✖
System Settings	✔	✖	✖	✖
Suspend Account	✔	✔	✖	✖
View Analytics	✔	✔	✔	✖

3.2 Agent Operations Framework

3.2.1 Agent Capabilities

- **Customer Account Creation:** Limited profile creation with essential fields
- **Wallet Operations:** Instant deposit, withdrawal request processing
- **Ticket Management:** Purchase tickets on behalf of customers
- **Transaction History:** Full visibility of customer transactions
- **Commission Structure:** Configurable rate (Phase 2 implementation)

3.2.2 Anti-Fraud Safeguards

Safeguard	Limit/Control
Daily Deposit Limit	BDT 50,000 per agent
Per Transaction Limit	BDT 10,000 maximum
Withdrawal Cooling Period	24 hours for new customers
Velocity Check	Maximum 10 deposits/hour
Agent Bond Requirement	BDT 100,000 (Phase 2)
KYC Verification	Required for >BDT 50,000 monthly

4 Transparency & Fair Play Mechanisms

4.1 Verifiable Draw Protocol

```sql
CREATE TABLE audit_logs (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT NOT NULL,
    action VARCHAR(100) NOT NULL,
    model_type VARCHAR(100) NOT NULL,
    model_id BIGINT NOT NULL,
    before_state JSON,
    after_state JSON,
    ip_address VARCHAR(45),
    user_agent TEXT,
    metadata JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_model (model_type, model_id),
    INDEX idx_user (user_id),
    INDEX idx_action (action),
    INDEX idx_created (created_at),

    CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Listing 6: Audit Trail Implementation

4.2 Transparency Features

- **Draw Hash:** Unique SHA-256 hash stored per draw
- **Timestamp Verification:** NTP-synchronized server time
- **Public Archive:** 30-day minimum retention period
- **ID Masking:** User-XXXX format for public display
- **QR Verification:** Digital receipt with verification endpoint

5 Development & Deployment Roadmap

5.1 Phase 1 - MVP Delivery (15-20 Days)

Timeline	Component	Deliverables
Week 1: Foundation		
Days 1-2	Core Setup	Laravel installation, Breeze auth, Spatie roles
Days 3-4	Wallet System	Database design, transaction processing
Days 5-6	Draw Management	CRUD operations, ticket purchasing
Day 7	QA Cycle	Unit testing, bug fixes, optimization
Week 2: Core Features		
Days 8-9	Winner Logic	Selection algorithm, verification system
Days 10-11	Admin Panel	Dashboard, user management, reports
Days 12-13	Agent System	Customer creation, wallet operations
Days 14-15	Notifications	Database notifications, email templates
Week 3: Polish & Deployment		
Days 16-17	Security Audit	Penetration testing, vulnerability scan
Days 18-19	Load Testing	500 concurrent users, performance tuning
Day 20	Deployment	Production release, monitoring setup

5.2 Phase 2 - Enhanced Features (Post-MVP)

1. **Payment Gateway Integration:** SSLCommerz, bKash, Nagad
2. **Real-time System:** WebSocket implementation for live countdown
3. **Multi-tier Prizes:** Grand, 2nd, 3rd, and consolation prizes
4. **Agent Commission:** Automated commission calculation and settlement
5. **Mobile Application:** React Native cross-platform app
6. **Analytics Suite:** Advanced reporting with predictive modeling

6 Quality Assurance & Security Framework

6.1 Security Hardening Checklist

Security Control	Implementation Status
SQL Injection Prevention	✔Eloquent ORM
XSS Protection	✔Blade escaping
CSRF Protection	✔VerifyCsrfToken middleware
Rate Limiting	✔5 attempts/10 minutes
Session Security	✔HTTP-only, secure cookies
Password Policy	✔Min 8 chars, mixed case
HTTPS Enforcement	✔Force SSL middleware
Database Encryption	⚠At-rest encryption (Phase 2)
Automated Backups	✔Daily with 30-day retention
Audit Logging	✔Comprehensive event tracking

6.2 Performance Targets

Metric	Target	Monitoring Tool
Page Load Time	$<$ 2 seconds	Laravel Telescope
API Response Time	$<$ 500ms	New Relic
Concurrent Users	500+	K6 Load Testing
Draw Selection (100k tickets)	$<$ 1 second	Custom benchmark
Database Query Time	$<$ 100ms	Laravel Debugbar
System Uptime	99.9%	UptimeRobot

7 Compliance & Legal Framework

7.1 Required Documentation

Document	Purpose
Terms & Conditions	User agreement, liability limitations
Privacy Policy	Data collection, GDPR compliance
Cookie Policy	Browser storage disclosure
Responsible Gaming Policy	Self-exclusion, deposit limits
Refund Policy	Ticket cancellation terms
AML/KYC Policy	Anti-money laundering procedures

7.2 Legal Terminology Guidelines

Restricted Term	Recommended Term	Rationale
Lottery	Raffle / Promotional Draw	Legal classification
Ticket Price	Entry Fee	Avoid gambling connotations
Winnings	Prize Credit	Tax implications
Players	Participants	Regulatory compliance
Gamble	Participate	Responsible messaging
Bet	Entry	Legal terminology

8 Risk Assessment & Mitigation

Risk Category	Probability	Impact	Mitigation Strategy
Duplicate Tickets	Low	High	Unique composite key, pessimistic locking
Balance Manipulation	Low	Critical	DB transactions, audit trail, idempotency
Draw Timing Issues	Medium	High	UTC storage, cron monitoring, manual override
Payment Fraud	Medium	High	Velocity checks, manual approval, IP tracking
Legal Compliance	Medium	Critical	Legal review, disclaimers, age verification
System Downtime	Low	High	Load balancing, failover, monitoring
Data Loss	Low	Critical	Daily backups, transaction logs
Winner Disputes	Medium	Medium	Verifiable seed, public hash, audit trail

9 Success Metrics & Key Performance Indicators

Metric	Target	Measurement Method
User Registration Completion	>80%	Funnel analytics
Ticket Purchase Conversion	>30%	Session tracking
Draw Participation Rate	>50%	Registered vs. participating
Admin Processing Time	<4 hours	Ticket resolution time
System Uptime	99.9%	Uptime monitoring
Critical Bugs	Zero	Sentry error tracking
User Satisfaction	>4.5/5	Post-interaction surveys
First Response Time	<2 hours	Support ticket metrics

10 Deployment & Environment Configuration

10.1 Production Environment Specifications

```
# Server Specifications
OS: Ubuntu 22.04 LTS
Web Server: Nginx 1.22+
PHP: PHP 8.2+ with extensions:
  - BCMath
  - Ctype
  - Fileinfo
  - JSON
  - Mbstring
  - OpenSSL
  - PDO
  - Tokenizer
  - XML
  - Redis
Database: MySQL 8.0
Cache/Queue: Redis 7.0
Process Control: Supervisor
SSL: Let's Encrypt / Comodo

# Application Configuration
APP_NAME="LuckyDraw Pro"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://draw.example.com

# Cache Configuration
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis

# Security Configuration
SESSION_SECURE_COOKIE=true
SESSION_SAME_SITE=strict
```

Listing 7: Production Server Requirements

10.2 CI/CD Pipeline

- **Version Control:** Git with GitFlow branching strategy
- **Repository:** GitHub / GitLab private repository
- **Automated Testing:** GitHub Actions / GitLab CI
- **Deployment:** Laravel Envoyer / Deployer
- **Monitoring:** Laravel Telescope, Sentry, UptimeRobot

11 Conclusion & Next Steps

11.1 Immediate Actions Required

1. **Legal Consultation:** Verify compliance with local gambling regulations
2. **Infrastructure Setup:** Provision staging and production environments
3. **Team Assembly:** Allocate development resources
4. **Kick-off Meeting:** Finalize requirements and timeline

11.2 Deliverable Acceptance Criteria

- All core features implemented and tested
- Zero critical security vulnerabilities
- Documentation complete and reviewed
- Load test passed for 500 concurrent users
- UAT signed off by stakeholders

DOCUMENT APPROVAL & AUTHORIZATION

This technical specification document has been reviewed and approved by the undersigned members of the project execution committee. By signing below, each individual confirms their acceptance of the system architecture, development timeline, and deliverables outlined herein.

Kawser Ahmed

System Architect

Lead System Designer
Date: February 12, 2026

Azmayeen Sadique

Project Manager

Program Director
Date: February 12, 2026

Asaduzzaman Asad

Technical Lead

Core Architecture & Development

Date: February 12, 2026

Mahamudul Hasan Pavel

Quality Assurance

Testing | Compliance | Audit

Date: February 12, 2026

VERIFICATION STATUS

All technical specifications validated and approved for MVP development.

Approval Reference: LDP-MVP-2026-001

Document Version: 1.0.0 **Status:** FINAL

CONFIDENTIALITY NOTICE

This document contains proprietary and trade secret information of LuckyDraw Pro. Unauthorized distribution, reproduction, or disclosure of this material in whole or in part is strictly prohibited and may result in legal action under applicable copyright and trade secret laws.

Copyright © 2026 LuckyDraw Pro. All Rights Reserved.